PHASE 1 – PROJECT DESIGN INTRODUCTION

Project Title: EduGenie – Google Gemini Powered Learning Assistant

The Project Design Phase converts the requirements identified earlier into a structured technical and user-interface design for EduGenie.

Design Objective: Create a simple, interactive, maintainable and scalable learning-assistant system that accepts student questions and displays AI-generated educational responses.

Design Principles: Simplicity, usability, modularity, secure credential handling, clear input/output flow, and easy future enhancement.

Design Outcome: A clear blueprint for implementing, testing and demonstrating the EduGenie application.

Evidence Placeholder: Insert the initial project design/planning screenshot here.

PHASE 2 – SYSTEM ARCHITECTURE DESIGN

The EduGenie architecture is based on an input-processing-output workflow.

Architecture Flow: User → User Interface → Application Logic → Gemini Service/API → Generated Response → User Interface.

User Interface Layer: Provides the question input field, submit/send control, and response display area.

Application Layer: Receives the user question, prepares the request, communicates with the configured AI service, and handles the returned response.

AI Service Layer: Google Gemini/Generative AI processes the prompt and generates an appropriate response according to the configured model and instructions.

Output Layer: The generated response is presented to the user in a readable format.

Evidence Placeholder: Insert the system architecture diagram here.

PHASE 3 – MODULE DESIGN

EduGenie can be organized into the following logical modules:

1. User Input Module – accepts the student question or learning prompt.

2. Validation Module – checks basic input conditions such as empty input where applicable.

3. Prompt/Request Module – prepares the request for the Gemini service.

4. Gemini Integration Module – sends the request and receives the generated response.

5. Response Handling Module – processes the returned response for display.

6. User Interface Module – presents input controls and generated answers.

7. Error Handling Module – handles normal request, network, or service errors.

Modular design makes the application easier to maintain and extend.

Evidence Placeholder: Insert a module/block diagram here.

PHASE 4 – USER INTERFACE DESIGN

The interface is designed to provide a simple learning experience with minimum navigation.

Proposed UI Elements:

• Project title: EduGenie – Google Gemini Powered Learning Assistant.

• Question/input box for entering a learning query.

• Submit/Send button for sending the request.

• Response area for displaying the generated answer.

• Optional clear/reset control if implemented.

UI Design Considerations: Readable text, clear labels, sufficient spacing, simple navigation, and visible feedback after submission.

The interface should make the input-to-response workflow easy to understand for students.

Evidence Placeholder: Insert the EduGenie UI wireframe or actual interface screenshot here.

PHASE 5 – DATA FLOW & PROCESS DESIGN

The main data flow begins when a student enters an educational question.

Step 1: User enters a question in the input field.

Step 2: Application validates the input as required.

Step 3: Application prepares the prompt/request.

Step 4: Request is sent to the configured Gemini service.

Step 5: Gemini generates a response.

Step 6: Application receives and processes the response.

Step 7: Response is displayed to the student.

Data Flow Summary: Question → Validation → Prompt → Gemini → Response → Display.

Evidence Placeholder: Insert a data-flow diagram or flowchart here.

PHASE 6 – DATA & TECHNOLOGY DESIGN

Data Design: The core interaction uses a user question as input and an AI-generated response as output. Any project configuration should be handled securely.

Technology Design:

• Programming language/environment used by the project.

• Google Gemini/Generative AI service for AI response generation.

• Required libraries and dependencies.

• Web browser or application interface, depending on the implementation.

• Development environment/IDE used by the team.

The exact versions and libraries should match the actual project environment.

Credential Design: API keys and tokens should be stored securely and never be included in screenshots or public documentation.

Evidence Placeholder: Insert technology-stack diagram or development-environment screenshot here.

PHASE 7 – SECURITY, ERROR & TEST DESIGN

Security Design: Keep API keys, passwords and tokens private. Use an appropriate secure configuration method for credentials.

Error Design:

• Empty input should be handled appropriately.

• Network/service failures should show a suitable user message.

• Unexpected responses should not expose sensitive technical information.

Test Design: Prepare test cases for valid educational questions, empty input, repeated questions, and normal service-error conditions.

Usability Design: The user should always understand where to enter a question and where the answer will appear.

Acceptance Design: The system is considered ready for demonstration when the complete input-to-output workflow works under normal conditions.

Evidence Placeholder: Insert security/error-flow or test-design diagram here.

PHASE 8 – FINAL DESIGN SUMMARY

Final Design: EduGenie is designed as a modular AI-powered learning assistant with a simple user interface and a clear input-to-response workflow.

Design Components: User Interface, Input Validation, Request/Prompt Handling, Gemini Integration, Response Handling, Error Handling, and Secure Configuration.

Key Design Benefits: Simple interaction, clear architecture, modular implementation, maintainability, and scope for future expansion.

Future Design Extensions: Subject-wise learning modes, quiz generation, multilingual support, voice interaction, progress tracking, and personalized learning features.

Final Conclusion: The Project Design Phase provides the technical and visual blueprint required to implement EduGenie effectively. The design can be used as a reference during coding, testing and final demonstration.

Final Evidence Placeholder: Insert the completed design/architecture diagram or final UI screenshot here.